Deterministic Where It Counts: Goals, Envelopes, and Evidence for Language-Model Agents in Industrial Operations*

Pieter van Schalkwyk
ORCID: 0009-0009-3982-9346
pieter.vanschalkwyk@xmpro.com

Gavin Green
ORCID: 0009-0007-8315-5559
gavin.green@xmpro.com

XMPro Inc.

Working paper, 19 October 2026

Abstract

Manufacturers in regulated and safety-critical industries increasingly ask for "deterministic" artificial intelligence (AI) agents. The request bundles three properties: reproducible output, a reproducible decision about whether a proposed action is allowed, and a decision that can be reconstructed after the fact. A hosted large language model (LLM) service should not be assumed to provide the first, and an industrial agent does not need it to. This paper gives an engineering account of where determinism can and should be built into an agent system that contains a language model. We make four contributions. First, we define the three properties formally, including the conditions under which each holds. Second, we argue that the agent's objective, expressed as utility and objective functions over measured plant values, fixes how success is evaluated: the model may propose different plans, but it cannot redefine success, while measured change still has to be attributed with care before it is credited to an action. Third, we specify the decision envelope that makes the decision to act reproducible for complete recorded inputs: an input envelope that validates and classifies observations before an agent sees them, and an action envelope that authorizes each request against current state at execution, with fail-safe authority, authoritative reads, complete coverage, application-time checks, and correlated confirmation, and with binding rules held as computable policy rather than as prompt text. Fourth, we propose an assurance protocol of five tests, including an objective-performance test. We state design requirements informed by a petrochemical deployment in which agent teams connected to the control system had passed most of their factory acceptance tests, locate the current enforcement boundary of the reference implementation, and relate the account to the draft EU GMP Annex 22 and to the 2026 discussion of its revision.

Keywords: industrial AI agents, determinism, objective functions, decision envelope, deterministic validation, assurance testing, multi-agent systems, governed autonomy, GMP, Annex 22

*Working paper, October 2026. Engineering companion to the governed decision-making paper (van Schalkwyk et al., 2026) and the functional-analogy account of reasoning (van Schalkwyk, 2026). Comments welcome.

1 Introduction

A request we now hear regularly from process and pharmaceutical manufacturers is for deterministic AI agents. It comes from engineers and quality leaders who have watched a language model give two different answers to the same question, and who work in industries where a batch record is an inspected document and a setpoint change has physical consequences. The request deserves a precise answer, because the word covers three different properties, and one of them should not be assumed of a hosted language model.

A theoretical foundation exists. A companion paper [28] argues, using Powell's framework for sequential decisions [23], that when safety-relevant decision logic is placed in a deterministic policy and the language model's output is treated as exogenous information, the governing decision can be reproduced from its recorded inputs, and replay provides conformance evidence for the recorded cases. A second companion paper [29] argues that the model's documented deficits, weak grounding, unfaithful reasoning traces, and brittleness under distribution shift, are best read as a requirements specification for the process harness around it.

This paper is the engineering sequel. It asks what has to be built, in an operating plant, for that result to hold in practice, how the resulting system should be judged, and how a buyer or regulator can test the claim. The answer has a positive part and a protective part. The positive part is the objective: an agent pursues a goal, and in operations that goal can be written down as a function over measured values that is evaluated the same way for the same inputs and configuration. The protective part is the decision envelope: rules on both sides of the agent that validate what it reads and authorize what it does.

The paper makes four contributions:

1. Formal definitions of the three properties called determinism, with the conditions under which each holds (Section 3).
2. A goal-referenced account of agent evaluation, in which the objective fixes how success is judged, constraints sit outside the objective, and measured change is distinguished from causal contribution (Section 4).
3. An engineering specification of the input and action envelopes, with computable policy as the form in which binding rules are held (Section 5).
4. An assurance protocol of five tests (Section 6).

Section 7 states design requirements informed by deployment work, Section 8 locates the current enforcement boundary of the reference implementation, XMPro MAGS (Multi-Agent Generative Systems), and Section 9 relates the account to pharmaceutical regulation and to the Industrial AI Agent Manifesto [1]. Statements about the reference implementation are tied to an inspected code version, and deployment status is dated; recommendations are stated as proposals.

2 Background and Related Work

What an agent is. Classical definitions describe an agent as a system that perceives its environment and acts on it in pursuit of goals [24, 30]. Gartner's current definition is close: AI agents are autonomous or semiautonomous software entities that use AI techniques to perceive, make decisions, take actions, and achieve goals in their digital or physical environments [8]. On either reading, a language model is one technique an agent may use, not the agent. On its own it produces an output from the inputs it is given; it does not obtain plant access, maintain operational goals, or execute actions. Much of the current confusion about agent determinism comes from treating the model and the agent as the same object.

Model-centric agents. A widely used design lets the language model drive control flow, interleaving reasoning with tool calls and deciding which action to take next and when to stop [31]. Each model-controlled choice can vary between runs, and the resulting proposal needs checking against the plant's current conditions. Cognitive architectures for language agents separate memory, internal actions, and external actions [25], which supports an alternative in which the model is called at defined points and ordinary software governs the rest. In the reference implementation, language-model provider code and prompt templates make up about 9% of the agent codebase, by a count on 17 October 2026 that removes blank lines and full-line comments; an earlier article reported 8% using a method not recorded here. A line count describes composition, not influence (Section 8).

Nondeterminism in inference. Hosted language-model services do not generally guarantee identical output for identical input, even at a sampling temperature of zero. One documented cause is that the numerical result of a request depends on how it is batched with other traffic; batch-invariant kernels restore reproducibility for self-hosted inference at a cost in throughput [9]. Reproducible model output is therefore possible under controlled inference conditions, but should not be assumed for a hosted service without a specific guarantee.

Independence and correlated error. Functional safety requires protection layers to be adequately independent of the control functions they protect [11]. In a classic experiment, independently developed versions of a program failed together far more often than an independence assumption predicts [16], and errors are strongly correlated across many contemporary language models [14]. Separate agents or models do not by themselves establish independent failure behavior; independence and common-cause failure must be assessed for the deployed architecture.

Human judgment and automation. Experts given identical cases often reach different conclusions [13]. Plants have never relied on operators agreeing; procedures, operating limits, permits, and interlocks determine what any operator may do. Human attention and working memory constrain monitoring performance [5], and alarm systems should support timely recognition and response [12]. Automation can be specified separately for information acquisition, analysis, decision selection, and action [21], and people asked to approve automated proposals are subject to automation bias [17, 22].

Regulation. The draft EU GMP Annex 22 limits AI in critical pharmaceutical manufacturing applications to static models with deterministic output and excludes generative AI and LLMs from them [3]. A 2026 European Medicines Agency report on an expert workshop records the drafting group discussing whether such models could be admitted under a documented, risk-based control strategy [4]. The US Food and Drug Administration's draft guidance frames AI credibility for regulatory decision-making in terms of model influence and decision consequence [6], and the two agencies have issued joint principles of good AI practice in drug development [7].

3 Three Meanings of Deterministic

Let x denote a component input, s_{exec} the complete state used by the authorization decision, p the parsed proposal, and c the approved configuration and implementation version: rules, thresholds, utility functions, prompts, model identity, and code. Three properties are commonly called determinism.

Reproducible output. A component f is output-deterministic if $x_1 = x_2$ implies $f(x_1) = f(x_2)$, where x includes every effective input, among them time values, external reads, and message ordering. Code satisfies this when those inputs are captured; so does a static validated

Table 1: Three properties called determinism.

Property	Achievable with an LLM in the loop
Reproducible output	Not to be assumed for a hosted model; possible under controlled inference; yes for code and static validated models with captured inputs
Deterministic disposition	Yes, when D is computed by deterministic checks over the complete execution state and configuration
Reconstructable decision	Yes, when the record holds everything D depended on

model executed under controlled conditions. A hosted language model should not be assumed to.

Deterministic disposition. The disposition

$$D(s_{\text{exec}}, p, c) \in \{\text{execute}, \text{refuse}, \text{escalate}\} \tag{1}$$

is deterministic when identical (s_{exec}, p, c) produce an identical result. The model may produce a different p on another run; for a given p, with the same execution state and configuration, D does not vary. State observed during planning can differ from state checked at execution, and both must be distinguished in the record. The authorization predicate of Section 5 decides whether a request may execute; a separate, versioned routing rule decides whether a denied or unsupported proposal is refused or escalated. Escalation never authorizes a plant write.

Reconstructable decision. A decision is reconstructable if there is a record ρ containing s_{exec}, p, c, and every value read by D, such that re-evaluating D on ρ reproduces the recorded disposition. Strictly this is auditability rather than determinism, but it is what an investigator means by the word.

We propose a practical definition for an industrial agent: *For a recorded execution state, proposal, and configuration, the system reaches the same disposition; and no request that fails the encoded checks is applied, whatever the model proposes.* Replay tests the first clause for recorded cases; regeneration and fault injection test the second across selected proposals and states (Section 6). A universal containment claim additionally requires an argument that every action path is covered and every check is correct. Neither clause requires the model to repeat itself.

4 The Objective Fixes How Success Is Judged

The definitions above say when a decision to act is reproducible. They do not say whether the act was any good. A system that repeats the same unsuccessful action is deterministic and of no value. Reproducibility of the disposition is necessary and not sufficient; the second half of the account is the objective.

4.1 Writing the goal down

Gartner's definition ends on goals [8]. In operations, the goal can be written precisely. Let m_i be the measured value of the i-th measure, $u_{i,c}$ a utility function under configuration c expressing how much each level of that measure is worth, and w_i its weight:

$$J_c(m) = \sum_i w_i \, u_{i,c}(m_i). \tag{2}$$

The approved configuration fixes the utility functions, weights, measurement definitions, and treatment of missing data. For the same measurements and configuration, J_c is the same every time. The language model can propose different plans. It cannot redefine success.

4.2 Constraints sit outside the objective

Safety and quality limits are not terms in J_c. Let $\mathcal{A}_0(c)$ be the set of well-formed actions defined in the configuration. The admissible actions in state s are

$$\mathcal{A}(s, c) = \{\, a \in \mathcal{A}_0(c) : g_k(s, a, c) \le 0 \text{ for all required } k \,\}. \tag{3}$$

The objective ranks only admissible actions. Within the mathematics, a limit expressed as a utility penalty can be traded away by a large enough gain elsewhere, and a limit expressed as a constraint cannot. An action is admissible only when every required check is defined and passes on sufficiently current inputs; missing, invalid, or unevaluated checks do not count as passes. This establishes permission under the encoded constraints. Physical safety additionally depends on the correctness and coverage of those constraints and on the assumptions of their execution.

4.3 Measured change and its attribution

Proposals carry predicted effects, which are useful for choosing among options. Over an evaluation horizon h agreed in advance, the measured change is

$$\Delta J = J_c(m_{t+h}) - J_c(m_t). \tag{4}$$

ΔJ describes the change; it does not by itself attribute that change to the agent's action, because disturbances and other interventions also move the measurements. The assurance protocol should record the predicted and measured change for each executed plan, record material disturbances and other interventions, and compare against a suitable baseline or controlled scenario before crediting a result to an action. Acceptance criteria may reward maintaining a target or preventing deterioration as well as improving the score. Agents should not be scored on their own prediction or self-assessment. A measure that becomes a target invites proxy optimization [18]; computing the objective from plant measurements outside the agent's control, and keeping hard limits outside the objective, reduce that risk without eliminating it.

4.4 Permitted and right are separate questions

Two operators on the same shift reason differently and may favor different moves. The plant accepts this because its rules define what is acceptable and its results, read with care, show which acceptable move worked better. The same separation applies to agents. The decision envelope (Section 5) constrains what may execute; whether a permitted action was the better one is assessed against the objective, using comparisons that account for disturbances. Figure 1 summarizes the analogy.

Worked example. Assume two runs of the same agent team, given the same state and evaluated under the same controlled disturbance scenario and horizon, propose different admissible actions: run A a small reduction in a reflux rate, run B a feed-rate trim. With illustrative values, A predicts $\widehat{\Delta J} = 0.040$ and achieves $\Delta J = 0.031$; B predicts 0.025 and achieves 0.034; both maintain every constraint margin. The runs behaved differently, as two operators might, and both actions were permitted. In this illustration B produces the larger measured improvement despite the lower prediction, which is evidence about prediction quality as well as about the choice. In an operating plant, attributing such a difference to the chosen actions would require further evidence.

5 The Decision Envelope

The decision envelope is the proposed arrangement of deterministic processing on both sides of the agent (Figure 2).

5.1 The input envelope

An agent reasoning over a failed transmitter reading can form a false view of the process, and output checks do not remove the need to validate its inputs. The input envelope validates observations and computes derived state outside the language model.

Quality classification. Each value reaching an agent carries an explicit source state (Table 2). A measurement with no approved classification rule is treated as bad. Validity is decided from the source state and never inferred from the numeric value; in particular, a missing value must never be coerced to zero, which in a zone calculation can place a measurement in a safe band. Frozen-value detection is a configured diagnostic, not infallible fault detection.

Substitution provenance must be carried separately from signal quality, so that a value substituted by the control system cannot reach an agent marked as live.

Derived state. Limits, zones, and operating mode are computed from approved tables and fixed rules and passed to the agent as interpreted state. For a valid finite measurement x with approved limits $L_{\text{high}} > L_{\text{low}}$, a normalized margin is

$$\mu = \frac{\min(x - L_{\text{low}},\ L_{\text{high}} - x)}{L_{\text{high}} - L_{\text{low}}}, \tag{5}$$

Table 2: Source states carried by each value reaching an agent.

State	Meaning
Live	Measured value passing its quality rules
Estimated	Replaced by a validated static model, such as a soft sensor
Held	Last good value, for a bounded period set per measurement class
Stale	Older than the permitted age for its source class, whether measured, estimated, or held
Bad	Failed quality rules, including a configured frozen-value diagnostic
Unknown	Insufficient information to classify

which reaches at most 0.5 inside the interval and is reported as unavailable when its inputs are invalid. It is recorded with its side and trend. Operating mode is derived by fixed priority and reported as unknown when any required input is bad. Where an agent cannot join several message streams, it receives one joined observation per cycle that also carries the authority state and the schema version.

5.2 The action envelope

A bounded vocabulary. Agents choose from a short list of engineered actions with typed parameters and enumerated or ranged values. Examples are approving or declining an operating band proposed by an existing optimizer, starting a sequence that has already been validated, writing a value within a mapped range, or escalating to an operator. Escalation is always available and never gated.

Authorization at execution. Each request r is authorized against the execution state immediately before it is applied:

$$\begin{aligned}\text{allow}(r, s) = {} & [r \in \mathcal{A}_0(c)] \wedge [\text{auth}(s) = \text{act}] \wedge [\text{age}(r) \le \tau] \\ & \wedge [\text{tgt}(r) \in T] \wedge [\text{val}(r) \in R(\text{tgt}(r))] \\ & \wedge [\text{lim}(r, s) = \text{pass}],\end{aligned} \tag{6}$$

where T is the set of targets agents may write to, R the permitted range for each, τ the maximum request age, and $\text{lim}(r, s) = \text{pass}$ exactly when every required g_k is defined and satisfied on sufficiently current inputs. Any false term refuses the request, records the refusal, and notifies the operator. The authority states *advise* and *act* govern only the agents' permission to write; in the deployment discussed in Section 7 they are called open and closed loop, which should not be confused with the topology of the plant's control loops. We propose four properties that make the gate a guarantee rather than an intention:

- *Fail-safe authority.* The authority state is *act* only on an explicit, current setting by an accountable person; anything missing, stale, or unreadable is *advise*. A communications watchdog may return the system to *advise* and must never grant *act*.

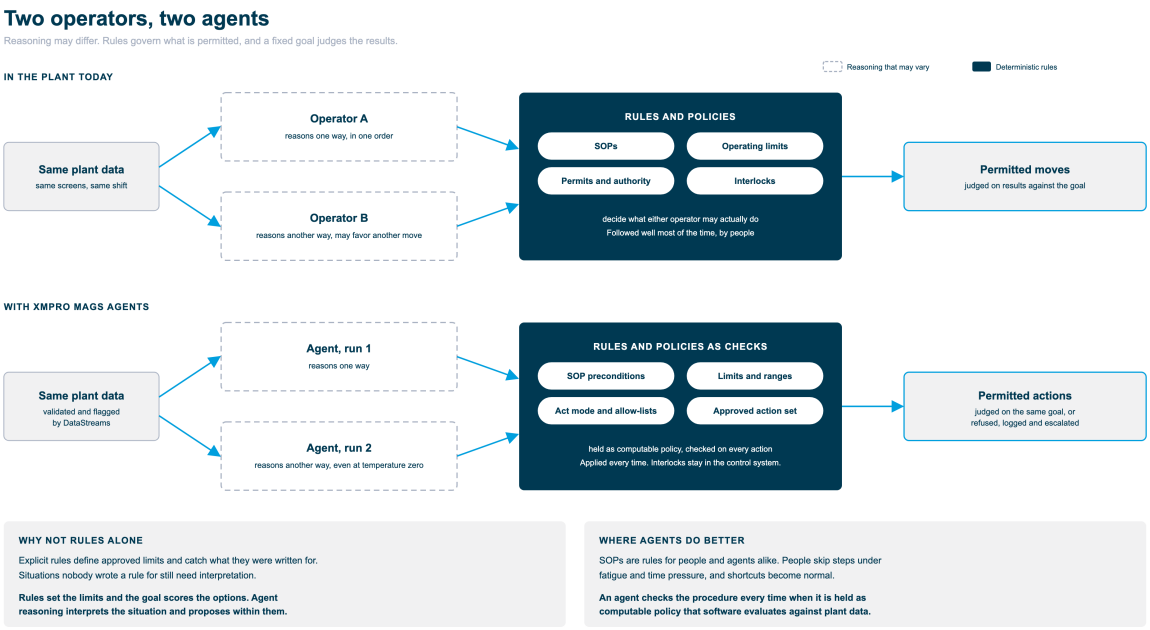


Figure 1: Two operators, or two runs of an agent, may propose different actions. Approved constraints govern permission, and a fixed objective evaluates measured results. Neither implies identical actions or identical plant outcomes.

- *Authoritative reads.* The gate reads the authority state from its source of record at execution, not from a cached or broadcast copy.
- *Complete coverage.* Every check that must hold is evaluated in every relevant action path. A deterministic check that returns a verdict only to the agent is advisory, however rigorous the check.
- *Assessed independence.* The control system's own logic decides whether to apply a value, and safety instrumented systems and interlocks remain separate from the agents and the gate. The adequacy of that separation, including shared infrastructure, credentials, and data paths, is assessed for the deployment [11].

Approval to application. A request waiting in a buffer can outlive the conditions it was approved under. At application, control-side logic verifies that the request remains authorized and that its required state predicates still hold. Buffered values are coupled to their authorization through a request or sequence identifier and a defined acknowledgment protocol. Behavior under delayed, reordered, repeated, and interrupted writes is verified; issuing a value before its enable flag is not by itself sufficient evidence of correct application.

Delivery and confirmation. Retry and duplicate handling are defined for each action, using request identity, expiry checks, and control-side deduplication or idempotent semantics where appropriate [15]. Transport delivery, command acceptance, and physical application are different properties. Publication, acceptance, application, and outcome are recorded separately, and application acknowledgments are correlated with the request; a readback showing the requested value does not by itself prove that this request caused it.

Table 3: Where rules can live, and what each placement guarantees.

Mechanism	Effect	Guarantee
Instruction in the prompt	May improve consistency of proposals	None
Deterministic check called by the agent	Informs the proposal	None unless also enforced in the action path
Deterministic check in every action path	Refuses non-compliant requests	Within its coverage and correctness
Control logic, interlocks, safety systems	Separate protection	Subject to assessed independence

5.3 Prompts and checks

The companion account of reasoning argues that decisions must not be adjudicated by the model's own rationale, because chain-of-thought explanations can rationalize choices driven by factors they never mention [26, 29]. A similar caution applies to instructions. An instruction in a prompt is something a model is asked to follow and may not; it may make proposals more consistent, and it guarantees nothing (Table 3).

The design rule follows: *Any mandatory precondition or limit for an agent action is implemented as a check in every*

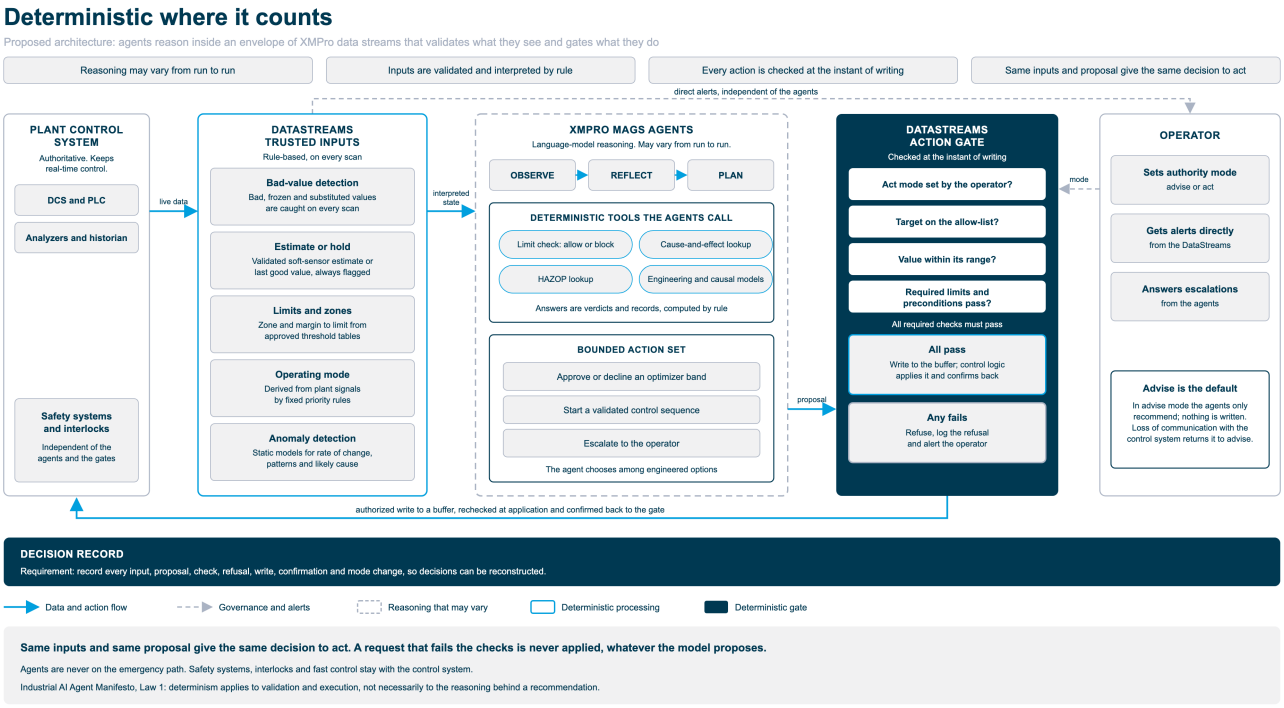


Figure 2: The decision envelope, as a proposed architecture and set of verification requirements. Data streams validate and interpret inputs by rule before an agent sees them; the agent's reasoning may vary from run to run; each proposed action is authorized against current state at the moment of writing, including all required process constraints and preconditions; application is confirmed against the request; the operator sets the authority state and receives alerts directly from the data streams.

relevant action path. Standard operating procedures belong to this rule layer. For an agent, their preconditions bind only when they are executed as checks; where they are, the check runs on every covered action and every shift, without the fatigue, time pressure, or normalized shortcuts that affect human compliance. The coverage and correctness of such checks still require testing.

5.4 Computable policy

The design rule above has a name in our earlier work: computable policy [20, 27]. A policy is computable when it is expressed in a form that software can evaluate against current state with an unambiguous result, rather than as prose for a person or a model to interpret. Deontic logic supplies the vocabulary of obligations, permissions, and prohibitions conditioned on context, and representation standards for it exist [10]. A computable policy rule can be written as

$$\pi = (\mu, \alpha, \varphi, \sigma, \kappa), \tag{7}$$

where $\mu \in \{\text{obligation}, \text{permission}, \text{prohibition}\}$ is the modality, α the class of actions it governs, $\varphi(s)$ a condition over measurable state, σ its scope and the authority that owns it, and κ its version. Evaluated against an execution state, each rule returns satisfied, violated, or not evaluable, and not evaluable never counts as satisfied. Prohibitions and unmet obligations become required checks g_k in the admissible set of Section 4; permissions define $\mathcal{A}_0(c)$ and the authority under which a request may proceed. Regulatory and organizational policies can be translated into enforceable form in the same way [19], and standard operating procedures become computable when their preconditions and step order are extracted into it.

Three consequences follow. A computable policy can be evaluated by the gate in every action path, where a policy in a prompt can only be read as advice. It can be versioned and placed under change control as configuration, so the record identifies which policy version authorized each action. And it can be reviewed directly by the people accountable for it, such as process engineers, safety managers, and quality units, in a form that does not depend on how a language model interprets prose. The same rules may also be rendered into an agent's prompt so that its proposals are more likely to comply, but the computable form is the one that binds. The Industrial AI Agent Manifesto names a deontic framework of obligations, prohibitions, permissions, and conditional duties, and the W3C Open Digital Rights Language (ODRL) as a representation for it [1, 10].

5.5 The decision record

Reconstruction requires a record of everything the deterministic path depended on: inputs with source states and timestamps, the derived state presented to the agent,

and the relevant external reads, time values, and message ordering; model identity, version, and the sampling settings actually used; the versions of prompts, rules, thresholds, and configuration, identified by content rather than name; the raw model output and the parsed proposal; each check evaluated, with the values read and its result; the authorization decision, the request identity, and the publication, acceptance, application, and outcome records; and the predicted effect beside the measured change over the evaluation horizon.

6 An Assurance Protocol

Reproducibility and usefulness are both claims, and each needs evidence. We propose five tests (Table 4).

The tests provide evidence, not proof. Replay demonstrates reproducibility for the recorded cases; regeneration and fault injection test containment across the proposals and states selected. Universal containment claims additionally require an argument about complete action-path coverage and check correctness. A system can refuse every request and still pass the containment tests; the objective test establishes whether it meets the agreed operating objective, and refusal alone does not establish process safety. Negative tests are particularly valuable when they show, from the control system's side as well as the agent system's log, that nothing was applied when the authority state or a check said no. Controlled comparisons can use a process benchmark such as the Tennessee Eastman problem [2] alongside historical replay, which can test detection and disposition but cannot show what an unexecuted alternative would have achieved.

7 Design Requirements Informed by Deployment Work

The design work draws on a petrochemical deployment in which agent teams are connected to the control system and, as confirmed on 17 October 2026, most factory acceptance tests had passed. The documented design uses input processing by data streams and an execution boundary around a bounded action set, with an operator-set authority state. The full envelope specified in this paper remains a set of verification requirements; the acceptance-test status does not establish that every property described here has been demonstrated. The deployment is not identified and no plant configuration is reported. The following requirements generalize from that work.

Coverage must be tested. A verdict constrains execution only when every relevant action path enforces it. Coverage should be tested directly rather than inferred from the existence of a check.

Absence must fail visibly. Missing thresholds must produce an explicit unavailable state, and an absent required limit must never produce an implicit pass.

Provenance must survive substitution. Substitution provenance is carried separately from signal quality, so that an estimated or substituted value cannot be mistaken for a live measurement.

Publish is not apply. Interfaces and records distinguish publication, acceptance, application, and confirmed outcome.

Approval must hold at application. Tests show that delayed application cannot use expired authorization or mismatched buffered values.

Configuration moves together. Action definitions, constraints, documentation, and enforcement points are versioned and reviewed together.

Prompts are measured separately from enforcement. Prompt instructions may reduce invalid proposals; that effect is measured separately from enforcement, which remains in the action path.

8 The Current Enforcement Boundary in MAGS

The companion paper locates safety-relevant logic in a deterministic policy [28]. In the MAGS baseline inspected at commit `938492a`, the arrangement is as follows.

- The language model is called at defined points in the observe, reflect, plan, and act cycle. Memory, significance and confidence calculation, objective scoring, consensus between agents, message routing, scheduling, and decision records are implemented in ordinary software. Reproducing their results requires capturing their complete effective inputs, including model-produced values, database results, relevant time values, message ordering, and configuration; being outside the model does not by itself establish end-to-end reproducibility.
- Utility functions and calculated measures are computed deterministically by data streams, and agents read the results. The planning score combines available proposal impacts with existing measure and utility values: where a latest utility value is available it takes precedence over the projected raw value, and missing impacts may use current values. It should not be described as a separately calculated prediction of realized objective performance; the predicted-versus-measured comparison proposed here requires explicit implementation and verification.
- MAGS records advisory policy-constraint evaluations during planning and proceeds when warnings are present. Deontic rules are supplied as prompt text, and approval-required communication rules do not yet implement an execution approval gate. In the terms of Section 5, these are policies that are not yet computable at the action boundary: the corresponding requirement is a computable form of each deontic rule

Table 4: Assurance tests for a governed agent system.

Test	Property	Pass condition
Reconstruction	Reproducible disposition for recorded cases	Re-evaluating recorded decisions with their recorded execution state, proposals, and configuration reproduces every disposition; any mismatch is a defect in the record or the path
Regeneration	Containment across selected proposals	For frozen states, including states near limits, every regenerated proposal is refused or admissible under the encoded checks; sequences of actions are tested as well as single actions
Objective	Operational usefulness	Against acceptance criteria and a horizon agreed before the trial, measured change meets the criteria relative to a suitable baseline or controlled scenario; predicted and measured effects are compared; refusals, escalations, and unmet objectives are recorded
Input fault injection	Input envelope	Bad, frozen, substituted, and stale readings produce the specified source state and the specified change in agent authority
Authority and timing	Action envelope	Revoked, expired, unauthorized, and duplicate requests produce no prohibited application; interruptions leave the system in its specified fallback state; control-side evidence agrees with the request record

and policy constraint, evaluated in every action path. Deployment-specific data streams enforce the checks configured in their action paths; policy constraints inside MAGS are not automatically transferred to them. An enforcement claim must therefore identify the particular constraint, its action-path check, and evidence that no action path bypasses it.

- The record schemas provide fields for model content, identity, token metrics, and integrity information, some of them optional. Complete decision replay requires verifying which fields each execution path populates and capturing the additional state, configuration, timing, and application evidence specified in Section 5.

The 9% line-count figure therefore describes composition, not the share of each decision the model influences: deterministic code downstream of the model, such as plan scoring, can operate on numbers the model proposed.

9 Regulatory and Standards Alignment

Pharmaceutical GMP. The draft Annex 22 restricts critical applications to static models whose output is identical for identical input and excludes generative AI and LLMs from them. It calls for a documented intended use, acceptance criteria set by a process expert before testing and "at least as high as the performance of the process it replaces", independent test data, appropriate confidence thresholds with consideration of an "undecided" outcome at very low confidence, and change, configuration, and performance control in operation [3]. The 2026 workshop report records industry arguing that acceptability should depend on intended use, decision consequence, model influence, uncertainty, complexity, and detectability rather than on a technology label, and the drafting group discussing admission of probabilistic and generative models under a fully documented, risk-based control strategy [4]. It describes layered controls for prevention, detection, and containment, including hard constraints, confidence gates, and fallback to validated states. The draft text is unchanged and the report is not a decision.

Table 5: Correspondence with the Annex 22 discussion.

Regulatory element	Element of this account
Intended use, model influence	Checks restrict authority; remaining influence assessed per decision
Acceptance criteria before testing	Objective function for performance criteria; separate tests for other criteria; objective test
Prevention, detection, containment	Input envelope; confidence handling; action envelope
Fallback to a validated state	Advise mode as one component, with the resulting process state assessed
Oversight, accountability	Operator-set authority state; escalation
Lifecycle control	Versioned models, prompts, rules, and thresholds under change control
Evidence package	Decision record; reconstruction test

The account relates to that discussion as summarized in Table 5. Deterministic checks restrict the model's authority to act; its remaining influence on which permitted action occurs must still be assessed for each intended use and decision. The objective function expresses agreed performance criteria, while safety, quality, data integrity, oversight, and failure-handling criteria require separate tests. Disabling agent writes is one possible component of a fallback; the resulting process state and the required operator response must be assessed and validated for the intended use. Replaying the deterministic authorization path can provide verification evidence when its complete inputs and configuration have been retained.

The Industrial AI Agent Manifesto. Law 1 places determinism in the validation and execution layer rather than in the reasoning behind a recommendation [1]. The

decision envelope is one engineering realization of that law, and the reconstruction and regeneration tests are candidate conformance evidence. This paper interprets reproducibility conditionally on the recorded proposal as well as the operational state; it does not claim that regenerating a proposal from the same plant state must select the same action. A law-by-law mapping appears in the companion paper [28].

10 Limitations and Open Questions

The account is engineering argument and design experience, not a controlled evaluation. The assurance protocol has been specified but not run as a matched comparison against baseline agents, and the objective test requires acceptance criteria, baselines, and evaluation horizons agreed with operations for each application. Attributing measured change to an agent's action remains difficult in an operating plant. Objective functions can be mis-specified; constraints outside the objective limit the damage but do not make a poor objective good. Which record fields each execution path populates has to be verified before a replay claim is made. The independence of the action gate from the agent depends on deployment details that this paper does not assess. The effectiveness of human oversight, including automation bias in approval, needs its own evidence [22]. Finally, the regulatory text is in revision; Section 9 describes alignment with a direction of discussion, not compliance with a final requirement.

11 Conclusion

A hosted language model should not be assumed to give the same answer twice, and an industrial agent does not need it to. What has to be fixed is what the agent reads, what it is allowed to do, the goal it is judged against, and the record that lets a decision be rebuilt. The objective function supplies the positive half: a goal evaluated the same way for the same inputs and configuration, which the model can pursue but cannot redefine. The decision envelope supplies the protective half: inputs validated by rule and actions authorized against current state at execution and again at application. The assurance protocol turns both into evidence a buyer or inspector can examine. On this account, the useful question for an operating organization is not whether its AI is deterministic, but which of its decisions must be reproducible, what objective each is judged against, and who holds the authority to let agents act.

References

[1] van Schalkwyk, P. (2026). The Industrial AI Agent Manifesto: The Ten Laws for Trustworthy Autonomous Operations. *Digital Twin Consortium Tech Brief*, 25 February 2026. URL: `https://xmpro.com/wp-content/uploads/2026/05/The-Industrial-AI-Agent-Manifesto.pdf`.

[2] Downs, J. J., & Vogel, E. F. (1993). A plant-wide industrial process control problem. *Computers & Chemical Engineering*, 17(3), 245–255. DOI: 10.1016/0098-1354(93)80018-I.

[3] European Commission & PIC/S (2025). EudraLex Volume 4, Annex 22: Artificial Intelligence. Stakeholders' consultation draft, July 2025.

[4] European Medicines Agency (2026). Multistakeholder workshop on expert contributions to artificial intelligence guidance development (Annex 22). Report EMA/156789/2026.

[5] Endsley, M. R. (1995). Toward a theory of situation awareness in dynamic systems. *Human Factors*, 37(1), 32–64. DOI: 10.1518/001872095779049543.

[6] U.S. Food and Drug Administration (2025). Considerations for the Use of Artificial Intelligence to Support Regulatory Decision-Making for Drug and Biological Products. Draft guidance, January 2025.

[7] U.S. Food and Drug Administration & European Medicines Agency (2026). Guiding Principles of Good AI Practice in Drug Development. January 2026.

[8] Gartner (2025). Gartner Hype Cycle Identifies Top AI Innovations in 2025. Press release, 5 August 2025.

[9] He, H., & Thinking Machines Lab (2025). Defeating nondeterminism in LLM inference. *Thinking Machines Lab: Connectionism*, September 2025. DOI: 10.64434/tml.20250910.

[10] Iannella, R., & Villata, S. (Eds.) (2018). ODRL Information Model 2.2. W3C Recommendation, 15 February 2018. URL: `https://www.w3.org/TR/2018/REC-odrl-model-20180215/`.

[11] International Electrotechnical Commission (2016). IEC 61511-1:2016, Functional safety: Safety instrumented systems for the process industry sector, Part 1: Framework, definitions, system, hardware and application programming requirements.

[12] International Electrotechnical Commission (2022). IEC 62682:2022, Management of alarm systems for the process industries.

[13] Kahneman, D., Sibony, O., & Sunstein, C. R. (2021). *Noise: A Flaw in Human Judgment*. Little, Brown Spark.

[14] Kim, E. M., Garg, A., Peng, K., & Garg, N. (2025). Correlated errors in large language models. *Proceedings of the 42nd International Conference on Machine Learning*, PMLR 267, 30038–30066. arXiv:2506.07962.

[15] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

[16] Knight, J. C., & Leveson, N. G. (1986). An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering*, SE-12(1), 96–109. DOI: 10.1109/TSE.1986.6312924.

[17] Lee, J. D., & See, K. A. (2004). Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1), 50–80. DOI: 10.1518/hfes.46.1.50_30392.

[18] Manheim, D., & Garrabrant, S. (2018). Categorizing variants of Goodhart's law. arXiv:1803.04585.

[19] Milosevic, Z., & Pyefinch, F. (2022). Computable consent: From regulatory, legislative, and organizational policies to security policies. *Enterprise Design, Operations, and Computing (EDOC 2022)*.

[20] Milosevic, Z., & van Schalkwyk, P. (2023). Towards responsible digital twins. *Enterprise Design, Operations, and Computing: EDOC 2023 Workshops*, 123–138.

[21] Parasuraman, R., Sheridan, T. B., & Wickens, C. D. (2000). A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics, Part A: Systems and Humans*, 30(3), 286–297. DOI: 10.1109/3468.844354.

[22] Parasuraman, R., & Manzey, D. H. (2010). Complacency and bias in human use of automation: An attentional integration. *Human Factors*, 52(3), 381–410. DOI: 10.1177/0018720810376055.

[23] Powell, W. B. (2022). *Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions*. Wiley. ISBN 9781119815037.

[24] Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

[25] Sumers, T. R., Yao, S., Narasimhan, K., & Griffiths, T. L. (2024). Cognitive architectures for language agents. *Transactions on Machine Learning Research*. arXiv:2309.02427.

[26] Turpin, M., Michael, J., Perez, E., & Bowman, S. R. (2023). Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. *Advances in Neural Information Processing Systems*, 36. arXiv:2305.04388.

[27] van Schalkwyk, P. (2025). Bounded autonomy: A pragmatic response to concerns about fully autonomous AI agents. *The Digital Engineer* (LinkedIn newsletter), 17 February 2025.

[28] van Schalkwyk, P., Green, G., Kontsos, A., Filippou, F., & Dannhauser, S. (2026). Governed decision-making for industrial multi-agent systems: A sequential decision analytics foundation for trustworthy and validatable autonomy. Preprint, June 2026.

[29] van Schalkwyk, P. (2026). Reasoning in governed LLM agents: A functional-analogy account of observation, reflection, and planning. Working paper, July 2026.

[30] Wooldridge, M. (2009). *An Introduction to MultiAgent Systems* (2nd ed.). Wiley.

[31] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*. arXiv:2210.03629.